

SMART CONTRACT

Security Audit Report

Project:	MagicDex
Platform:	Shido
Language:	Solidity
Date:	November 8th, 2024

Table of contents

Introduction	4
Project Background	4
Audit Scope	5
Claimed Smart Contract Features	6
Audit Summary.....	7
Technical Quick Stats.....	8
Code Quality.....	9
Documentation	9
Use of Dependencies.....	9
AS-IS overview.....	10
Severity Definitions.....	14
Audit Findings.....	15
Conclusion	20
Our Methodology	21
Disclaimers	23
Appendix	
• Solhint Linter	28

THIS IS SECURITY AUDIT REPORT DOCUMENT AND WHICH MAY CONTAIN INFORMATION WHICH IS CONFIDENTIAL. WHICH INCLUDES ANY POTENTIAL VULNERABILITIES AND MALICIOUS CODES WHICH CAN BE USED TO EXPLOIT THE SOFTWARE. THIS MUST BE REFERRED INTERNALLY AND ONLY SHOULD BE MADE AVAILABLE TO THE PUBLIC AFTER ISSUES ARE RESOLVED.

Introduction

EtherAuthority was contracted by the MagicDex team to perform the Security audit of the MagicDex smart contracts code. The audit has been performed using manual analysis as well as using automated software tools. This report presents all the findings regarding the audit performed on November 8th, 2024.

The purpose of this audit was to address the following:

- Ensure that all claimed functions exist and function correctly.
- Identify any security vulnerabilities that may be present in the smart contract.

Project Background

- MagicDex Smart Contracts is a contract that can be divided into multiples, each with unique functionalities:
 - **MagicToken:** This contract is utilized for transferring token, renouncing ownership, and transferring ownership.
 - **MagicFactory:** This contract is utilized for creating pair addresses, updating the fee address, and setting the fee to the setter address.
 - **MagicRouter:** This contract is utilized for updating liquidity and swapping tokens.
 - **MasterChef:** This contract updates the pool, updates the reward variables of the given pool to be up-to-date, deposits LP tokens to MasterChef for token point allocation, withdraws LP tokens, and performs emergency withdrawals.

Audit scope

Name	Code Review and Security Analysis Report for MagicDex Smart Contracts
Platform	Shido Chain
Laguage	Solidity
File 1	contract.sol
File 1 Initial code link	0xd9650144Cd028fc70270a26C1D878E5dbeF8CE9A
File 2	MagicFactory.sol
File 2 Initial code link	0x6f627b6967f196F5AcdB2913cc4B4751350ca1E2
File 3	MagicRouter.sol
File 3 Initial code link	0xAeFEbb82F665541Cdbdfa1D34F815bb1D8568Ff3
File 4	MasterChef.sol
File 4 Initial code link	0x76fB7D183383086f9bFb58c67396aae9Ac4b830f
Audit Date	November 8th, 2024

Claimed Smart Contract Features

Claimed Feature Detail	Our Observation
File 1 contract.sol • Name: MAGIC • Symbol: MAGIC • Decimals: 18 • Total Supply: 10 billion	YES, This is valid.
File 2 MagicFactory.sol <u>The Owner has control over the following functions:</u> • Update the fee address. • Update the fee to the setter address.	YES, This is valid.
File 3 MagicRouter.sol • This contract is utilized to update Liquidity.	YES, This is valid.
File 4 MasterChef.sol <u>The Owner has control over the following functions:</u> • The owner can update the multiplier value. • Update the given pool's token point allocation by the owner. • The owner can update the emission rate. • Fee address can be set by the owner. • StartBlock can be updated by the owner. • Current owner can transfer the ownership. • Owner can renounce ownership.	YES, This is valid.

Audit Summary

According to the standard audit assessment, Customer`s solidity smart contracts are **“Secured”**. Also, these contracts contain owner control, which does not make them fully decentralized.



We used various tools like Slither, Solhint and Remix IDE. At the same time this finding is based on critical analysis of the manual audit.

All issues found during automated analysis were manually reviewed and applicable vulnerabilities are presented in the Audit overview section. General overview is presented in AS-IS section and all identified issues can be found in the Audit overview section.

We found 0 critical, 0 high, 0 medium, 6 low and 5 very low level issues.

Investors Advice: Technical audit of the smart contract does not guarantee the ethical nature of the project. Any owner controlled functions should be executed by the owner with responsibility. All investors/users are advised to do their due diligence before investing in the project.

Technical Quick Stats

Main Category	Subcategory	Result
Contract Programming	Solidity version not specified	Passed
	Solidity version too old	Moderated
	Integer overflow/underflow	Passed
	Function input parameters lack of check	Passed
	Function input parameters check bypass	Passed
	Function access control lacks management	Passed
	Critical operation lacks event log	Moderated
	Human/contract checks bypass	Passed
	Random number generation/use vulnerability	N/A
	Fallback function misuse	Passed
	Race condition	Passed
	Logical vulnerability	Passed
	Features claimed	Passed
	Other programming issues	Moderated
Code Specification	Function visibility not explicitly declared	Passed
	Var. storage location not explicitly declared	Passed
	Use keywords/functions to be deprecated	Passed
	Unused code	Passed
Gas Optimization	"Out of Gas" Issue	Passed
	High consumption 'for/while' loop	Passed
	High consumption 'storage' storage	Passed
	Assert() misuse	Passed
Business Risk	The maximum limit for mintage not set	Moderated
	"Short Address" Attack	Passed
	"Double Spend" Attack	Passed

Overall Audit Result: **PASSED**

Code Quality

This audit scope has 4 smart contract files. Smart contracts contain Libraries, Smart contracts, inherits and Interfaces. This is a compact and well written smart contract.

The libraries in MagicDex are part of its logical algorithm. A library is a different type of smart contract that contains reusable code. Once deployed on the blockchain (only once), it is assigned a specific address and its properties / methods can be reused many times by other contracts in the MagicDex Protocol.

The MagicDex team has not provided unit test scripts, which would have helped to determine the integrity of the code in an automated way.

Code parts are not well commented on smart contracts.

Documentation

We were given a MagicDex smart contract code in the form of a shidoscan.com web link. The hash of that code is mentioned above in the table.

As mentioned above, code parts are not well commented on. But the logic is straightforward. So it is easy to quickly understand the programming flow as well as complex code logic. Comments are very helpful in understanding the overall architecture of the protocol.

Use of Dependencies

As per our observation, the libraries are used in this smart contracts infrastructure that are based on well known industry standard open source projects.

Apart from libraries, its functions are not used in external smart contract calls.

AS-IS overview

contract.sol

Functions

Sl.	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	delegates	external	Passed	No Issue
3	delegate	external	Passed	No Issue
4	delegateBySig	external	Passed	No Issue
5	getCurrentVotes	external	Passed	No Issue
6	getPriorVotes	external	Passed	No Issue
7	_delegate	internal	Passed	No Issue
8	_moveDelegates	internal	Passed	No Issue
9	_writeCheckpoint	internal	Passed	No Issue
10	safe32	internal	Passed	No Issue
11	getChainId	internal	Passed	No Issue
12	getOwner	external	Passed	No Issue
13	name	read	Passed	No Issue
14	decimals	read	Passed	No Issue
15	symbol	read	Passed	No Issue
16	totalSupply	read	Passed	No Issue
17	balanceOf	read	Passed	No Issue
18	transfer	write	Passed	No Issue
19	allowance	read	Passed	No Issue
20	approve	write	Passed	No Issue
21	transferFrom	write	Passed	No Issue
22	increaseAllowance	write	Passed	No Issue
23	decreaseAllowance	write	Passed	No Issue
24	_transfer	internal	Passed	No Issue
25	_approve	internal	Passed	No Issue
26	owner	read	Passed	No Issue
27	onlyOwner	modifier	Passed	No Issue
28	renounceOwnership	write	access only Owner	No Issue
29	transferOwnership	write	access only Owner	No Issue
30	_transferOwnership	internal	Passed	No Issue

MagicFactory.sol

Functions

Sl.	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	allPairsLength	external	Passed	No Issue
3	createPair	external	Passed	No Issue
4	setFeeTo	external	Passed	No Issue
5	setFeeToSetter	external	Passed	No Issue
6	lock	modifier	Passed	No Issue
7	getReserves	read	Passed	No Issue
8	_safeTransfer	write	Passed	No Issue
9	initialize	external	Passed	No Issue
10	_update	write	Passed	No Issue
11	_mintFee	write	Passed	No Issue
12	mint	external	Passed	No Issue
13	burn	external	lock	No Issue
14	swap	external	lock	No Issue
15	skim	external	lock	No Issue
16	sync	external	lock	No Issue
17	_mint	internal	Passed	No Issue
18	_burn	internal	Passed	No Issue
19	_approve	write	Passed	No Issue
20	_transfer	write	Passed	No Issue
21	approve	external	Passed	No Issue
22	transfer	external	Passed	No Issue
23	transferFrom	external	Passed	No Issue
24	permit	external	Passed	No Issue

MagicRouter.sol

Functions

Sl.	Functions	Type	Observation	Conclusion
1	constructor	write	Passed	No Issue
2	ensure	modifier	Passed	No Issue
3	receive	external	Passed	No Issue
4	_addLiquidity	internal	Passed	No Issue
5	addLiquidity	external	Passed	No Issue
6	addLiquidityETH	external	Passed	No Issue
7	removeLiquidity	write	Passed	No Issue
8	removeLiquidityETH	write	Passed	No Issue
9	removeLiquidityWithPermit	external	Passed	No Issue
10	removeLiquidityETHWithPermit	external	Passed	No Issue
11	removeLiquidityETHSupportingFeeOnTransferTokens	write	Passed	No Issue
12	removeLiquidityETHWithPermitSupportingFeeOnTransferTokens	external	Passed	No Issue

13	_swap	internal	Passed	No Issue
14	swapExactTokensForTokens	external	Passed	No Issue
15	swapTokensForExactTokens	external	Passed	No Issue
16	swapExactETHForTokens	external	Passed	No Issue
17	swapTokensForExactETH	external	Passed	No Issue
18	swapExactTokensForETH	external	Passed	No Issue
19	swapETHForExactTokens	external	Passed	No Issue
20	_swapSupportingFeeOnTransferTokens	internal	Passed	No Issue
21	swapExactTokensForTokensSupportingFeeOnTransferTokens	external	Passed	No Issue
22	swapExactETHForTokensSupportingFeeOnTransferTokens	external	Passed	No Issue
23	swapExactETHForTokensSupportingFeeOnTransferTokens	external	Passed	No Issue
24	swapExactTokensForETHSupportingFeeOnTransferTokens	external	Passed	No Issue
25	quote	write	Passed	No Issue
26	getAmountOut	write	Passed	No Issue
27	getAmountIn	write	Passed	No Issue
28	getAmountsOut	read	Passed	No Issue
29	getAmountsIn	read	Passed	No Issue

MasterChef.sol

Functions

Sl.	Functions	Type	Observation	Conclusion
1	constructor	write	Hardcoded BEP20 token address	Refer Audit Findings
2	updateMultiplier	write	Multiplier limit not set, Range validation missing	Refer Audit Findings
3	poolLength	external	Passed	No Issue
4	add	write	Critical operation lacks event log, Duplicate lp token	Refer Audit Findings
5	set	write	Critical operation lacks event log	Refer Audit Findings
6	updateStakingPool	internal	Passed	No Issue
7	getMultiplier	read	Passed	No Issue
8	pendingMagic	external	Passed	No Issue
9	massUpdatePools	write	Passed	No Issue
10	updatePool	write	Critical operation lacks event log	Refer Audit Findings
11	deposit	write	Change comments	Refer Audit Findings
12	withdraw	write	Passed	No Issue
13	enterStaking	write	Change comments	Refer Audit Findings
14	leaveStaking	write	Passed	No Issue

15	emergencyWithdraw	write	Passed	No Issue
16	safeMagicTransfer	internal	Passed	No Issue
17	updateEmissionRate	write	Critical operation lacks event log, EmissionRate limit not set	Refer Audit Findings
18	setFeeAddress	write	Passed	No Issue
19	updateStartBlock	external	access only Owner	No Issue
20	owner	read	Passed	No Issue
21	onlyOwner	modifier	Passed	No Issue
22	renounceOwnership	write	access only Owner	No Issue
23	transferOwnership	write	access only Owner	No Issue
24	_transferOwnership	internal	Passed	No Issue

Severity Definitions

Risk Level	Description
Critical	Critical vulnerabilities are usually straightforward to exploit and can lead to token loss etc.
High	High-level vulnerabilities are difficult to exploit; however, they also have significant impact on smart contract execution, e.g. public access to crucial
Medium	Medium-level vulnerabilities are important to fix; however, they can't lead to tokens lose
Low	Low-level vulnerabilities are mostly related to outdated, unused etc. code snippets, that can't have significant impact on execution
Lowest / Code Style / Best Practice	Lowest-level vulnerabilities, code style violations and info statements can't affect smart contract execution and can be ignored.

Audit Findings

Critical Severity

No critical severity vulnerabilities were found in the contract code.

High Severity

No high severity vulnerabilities were found in the contract code.

Medium

No medium severity vulnerabilities were found in the contract code.

Low

(1) Critical operation lacks event log: [Masterchef.sol](#)

Missing event log for:

- add
- set
- updatePool
- updateEmissionRate

Resolution: Write an event log for listed events.

(2) EmissionRate limit not set: [Masterchef.sol](#)

```
function updateEmissionRate(uint256 _magicPerBlock) public onlyOwner {  
    massUpdatePools();  
    emit EmissionRateUpdated(msg.sender, magicPerBlock, _magicPerBlock);  
    magicPerBlock = _magicPerBlock;  
}
```

In the updateEmissionRate function, the magicPerBlock variable can be set with any number value. No range limit is set.

Resolution: We suggest setting an explicit limit before assigning value.

(3) Multiplier limit not set: [Masterchef.sol](#)

```
function updateMultiplier(uint256 multiplierNumber) public onlyOwner {  
    BONUS_MULTIPLIER = multiplierNumber;  
}
```

In the updateMultiplier function, the BONUS_MULTIPLIER variable can be set with any number value. No range limit is set.

Resolution: We suggest setting an explicit limit before assigning value.

(4) Hardcoded BEP20 token address: [Masterchef.sol](#)

```
constructor(  
    MAGIC _magic,  
    address _feeAddr,  
    uint256 _magicPerBlock,  
    uint256 _startBlock  
) public {  
  
    magic = _magic;  
    feeAddr = _feeAddr;  
    magicPerBlock = _magicPerBlock;  
    startBlock = _startBlock;  
  
    // staking pool  
    poolInfo.push(PoolInfo({  
        lpToken: IBEP20(0xd9650144Cd028fc70270a26C1D878E5dbeF8CE9A),  
        allocPoint: 1000,  
        lastRewardBlock: startBlock,  
        accMAGICPerShare: 0,  
        depositFeeBP: 0  
    }));  
  
    totalAllocPoint = 1000;  
}
```

In constructor, while staking pool BEP20 token address is hardcoded.

Resolution: We suggest confirming the token address before deploying the contract.

(5) Duplicate lp token: [Masterchef.sol](#)

```
// Add a new lp to the pool. Can only be called by the owner.
// XXX DO NOT add the same LP token more than once. Rewards will be messed up if you do.
function add(uint256 _allocPoint, IBEP20 _lpToken, uint16 _depositFeeBP, bool _withUpdate) public onlyOwner {
    require(_depositFeeBP <= 1000, "add: invalid deposit fee basis points");
    if (_withUpdate) {
        massUpdatePools();
    }
    uint256 lastRewardBlock = block.number > startBlock ? block.number : startBlock;
    totalAllocPoint = totalAllocPoint.add(_allocPoint);
    poolInfo.push(PoolInfo({
        lpToken: _lpToken,
        allocPoint: _allocPoint,
        lastRewardBlock: lastRewardBlock,
        accMAGICPerShare: 0,
        depositFeeBP: _depositFeeBP
    }));
    updateStakingPool();
}
```

In the add function, users can add duplicate lp tokens.

Resolution: We suggest adding validation to check for duplicate lp tokens.

(6) Range validation missing: [Masterchef.sol](#)

```
function updateMultiplier(uint256 multiplierNumber) public onlyOwner {  infinite gas
    BONUS_MULTIPLIER = multiplierNumber;
}
```

In the updateMultiplier functions, The role can set the following state variables arbitrarily large or small causing potential risks in BONUS_MULTIPLIER.

Resolution: We recommend setting ranges and check the following input variable: multiplierNumber.

Very Low / Informational / Best practices:

(1) Use latest solidity version:

[Masterchef.sol](#)

```
pragma solidity 0.6.12;
```

[MagicRouter.sol](#)

```
// SPDX-License-Identifier: MIT
pragma solidity =0.6.6;
```

be disclosed to third party without prior written permission of EtherAuthority.

Email: audit@EtherAuthority.io

MagicFactory.sol

```
// SPDX-License-Identifier: MIT
pragma solidity =0.5.16;
```

Using the latest solidity will prevent any compiler level bugs.

Resolution: Please use 0.8.23 which is the latest version.

(2) Multiple pragma: [MasterChef.sol](#)

```
5 // SPDX-License-Identifier: MIT
6
7 pragma solidity 0.6.12;

955 pragma solidity ^0.6.0;
956
957 interface MAGIC {
958     function transfer(address recipient, uint256 amount) external returns (bool);
959     function balanceOf(address account) external view returns (uint256);
960     function transferFrom(address sender, address recipient, uint256 amount) external returns (bool);
961 }
```

There are multiple pragmas with different compiler versions.

Resolution: We suggest using only one pragma and removing the other.

(3) Parameter can be immutable: [MasterChef.sol](#)

```
993 // The OVE TOKEN!
994 MAGIC public magic;
```

Variables that are set within the constructor but further remain unchanged should be marked as immutable to save gas and to ease the reviewing process of third-parties.

Resolution: Consider marking this variable as immutable.

Centralization

This smart contract has some functions which can be executed by the Admin (Owner) only. If the admin wallet private key would be compromised, then it would create trouble.

Following are Admin functions:

MagicFactory.sol

- setFeeTo: Fee address can be set by the owner.
- setFeeToSetter: Fee to setter address can be set by the owner.

MagicPair.sol

- initialize: The owner can be called once by the factory at time of deployment.

MasterChef.sol

- updateMultiplier: The Owner can update the multiplier value.
- add: Add a new lp to the pool by the owner.
- set: Update the given pool's token point allocation by the owner.
- updateEmissionRate: The Owner can update the emission rate.
- setFeeAddress: Fee address can be set by the owner.
- updateStartBlock: StartBlock can be updated by the owner.

Ownable.sol

- renounce Ownership: Deleting ownership will leave the contract without an owner, removing any owner-only functionality.
- transfer ownership: The current owner can transfer ownership of the contract to a new account.

To make the smart contract 100% decentralized, we suggest renouncing ownership of the smart contract once its function is completed.

Conclusion

We were given a contract code in the form of a shidoscan.com web link. And we have used all possible tests based on given objects as files. We had observed 6 low and 5 Informational severity issues in the smart contracts, but those are not critical ones. **So, the smart contracts are ready for the mainnet deployment.**

Since possible test cases can be unlimited for such smart contracts protocol, we provide no such guarantee of future outcomes. We have used all the latest static tools and manual observations to cover maximum possible test cases to scan everything.

Smart contracts within the scope were manually reviewed and analyzed with static analysis tools. Smart Contract's high-level description of functionality was presented in the As-is overview section of the report.

The audit report contains all found security vulnerabilities and other issues in the reviewed code.

The security state of the reviewed contract, based on standard audit procedure scope, is **"Secured"**.

Our Methodology

We like to work with a transparent process and make our reviews a collaborative effort. The goals of our security audits are to improve the quality of systems we review and aim for sufficient remediation to help protect users. The following is the methodology we use in our security audit process.

Manual Code Review:

In manually reviewing all of the code, we look for any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behavior when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our audit techniques included manual code analysis, user interface interaction, and whitebox penetration testing. We look at the project's web site to get a high level understanding of what functionality the software under review provides. We then meet with the developers to gain an appreciation of their vision of the software. We install and use the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We follow a conservative, transparent process for analyzing potential security vulnerabilities and seeing them through successful remediation. Whenever a potential issue is discovered, we immediately create an Issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is conservative because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyze the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take, and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinized by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the details are made public.

Disclaimers

EtherAuthority.io Disclaimer

EtherAuthority team has analyzed this smart contract in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment and functionality (performing the intended functions).

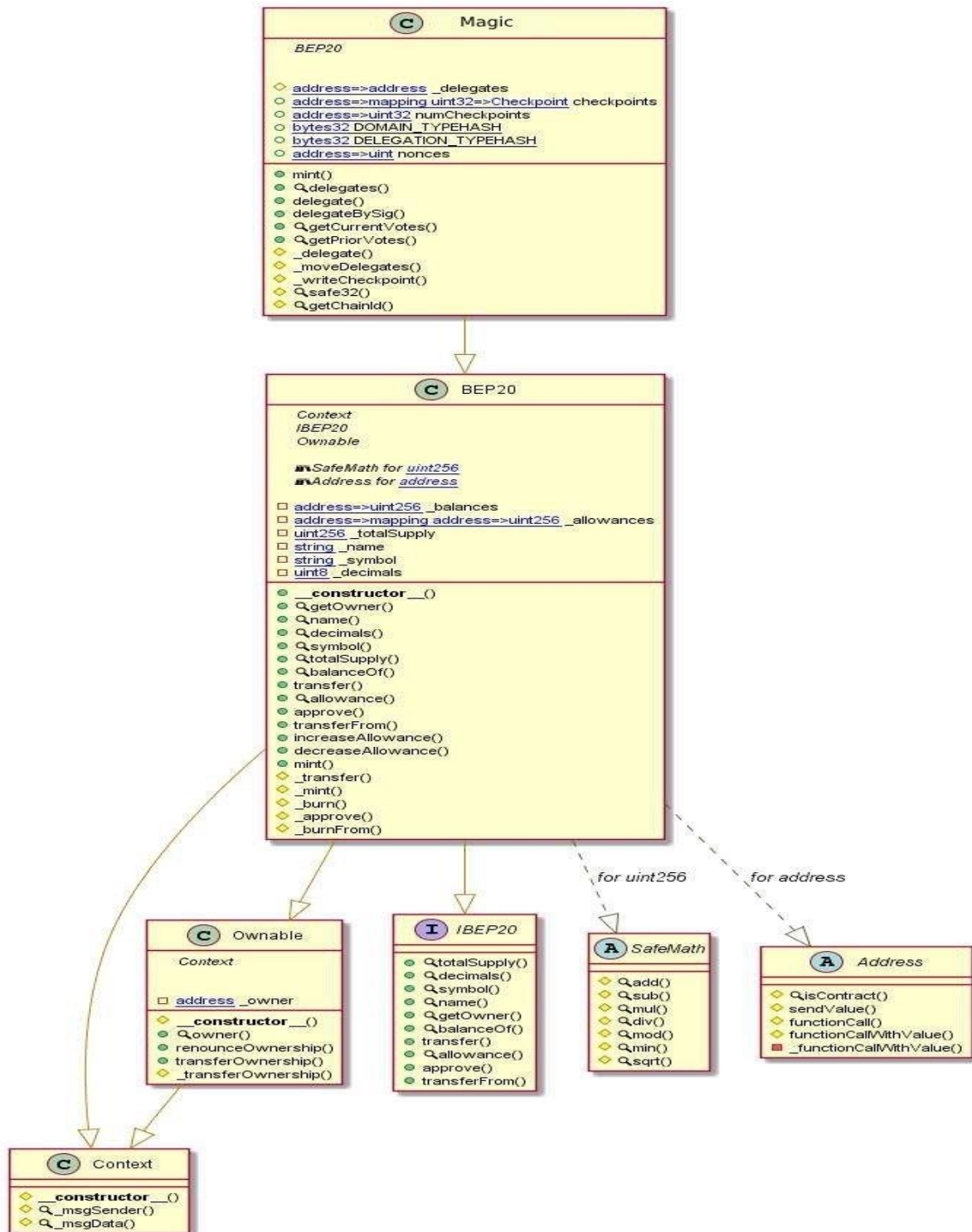
Due to the fact that the total number of test cases are unlimited, the audit makes no statements or warranties on security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bugfree status or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Technical Disclaimer

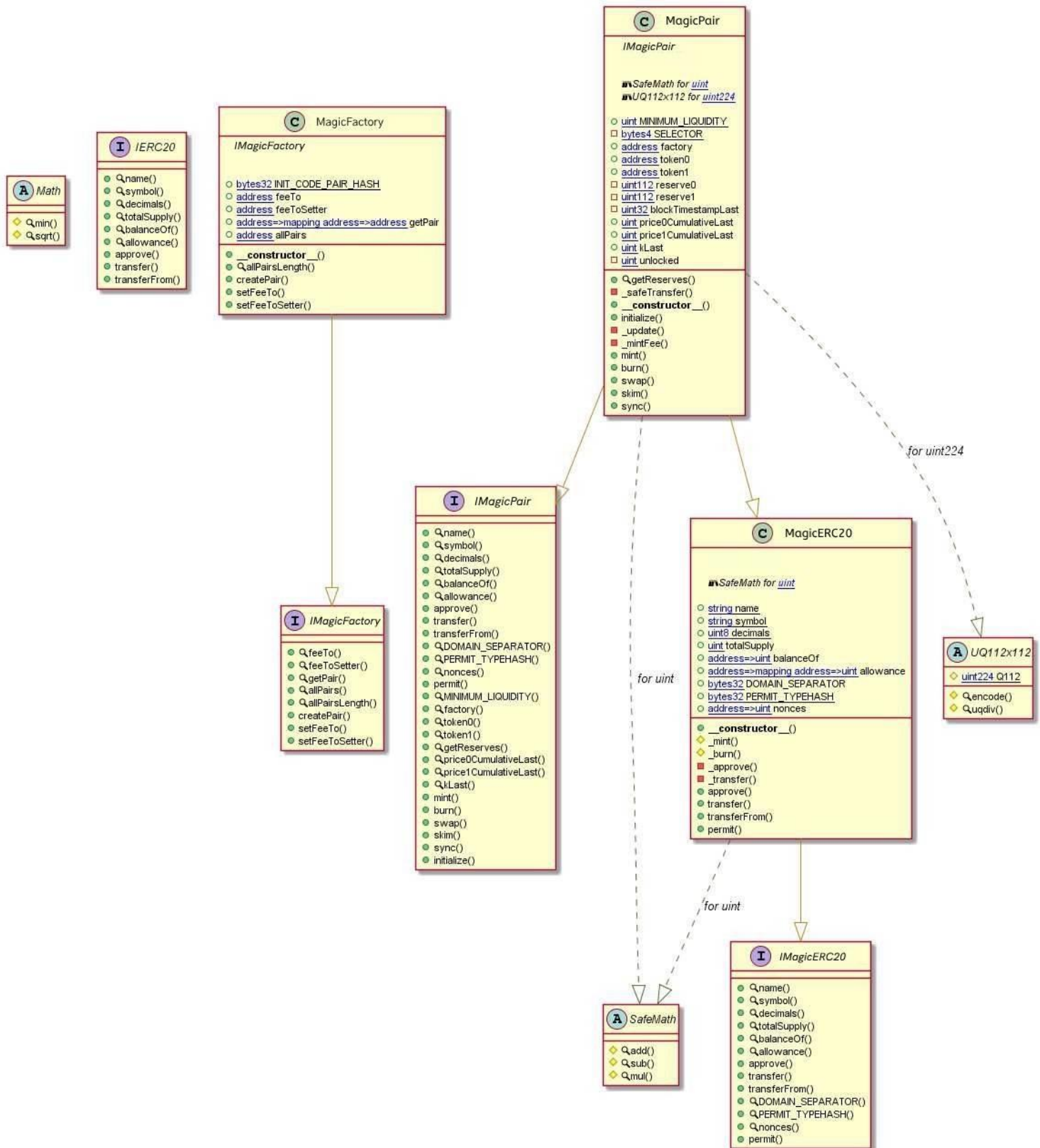
Smart contracts are deployed and executed on the blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to hacks. Thus, the audit can't guarantee explicit security of the audited smart contracts.

Appendix

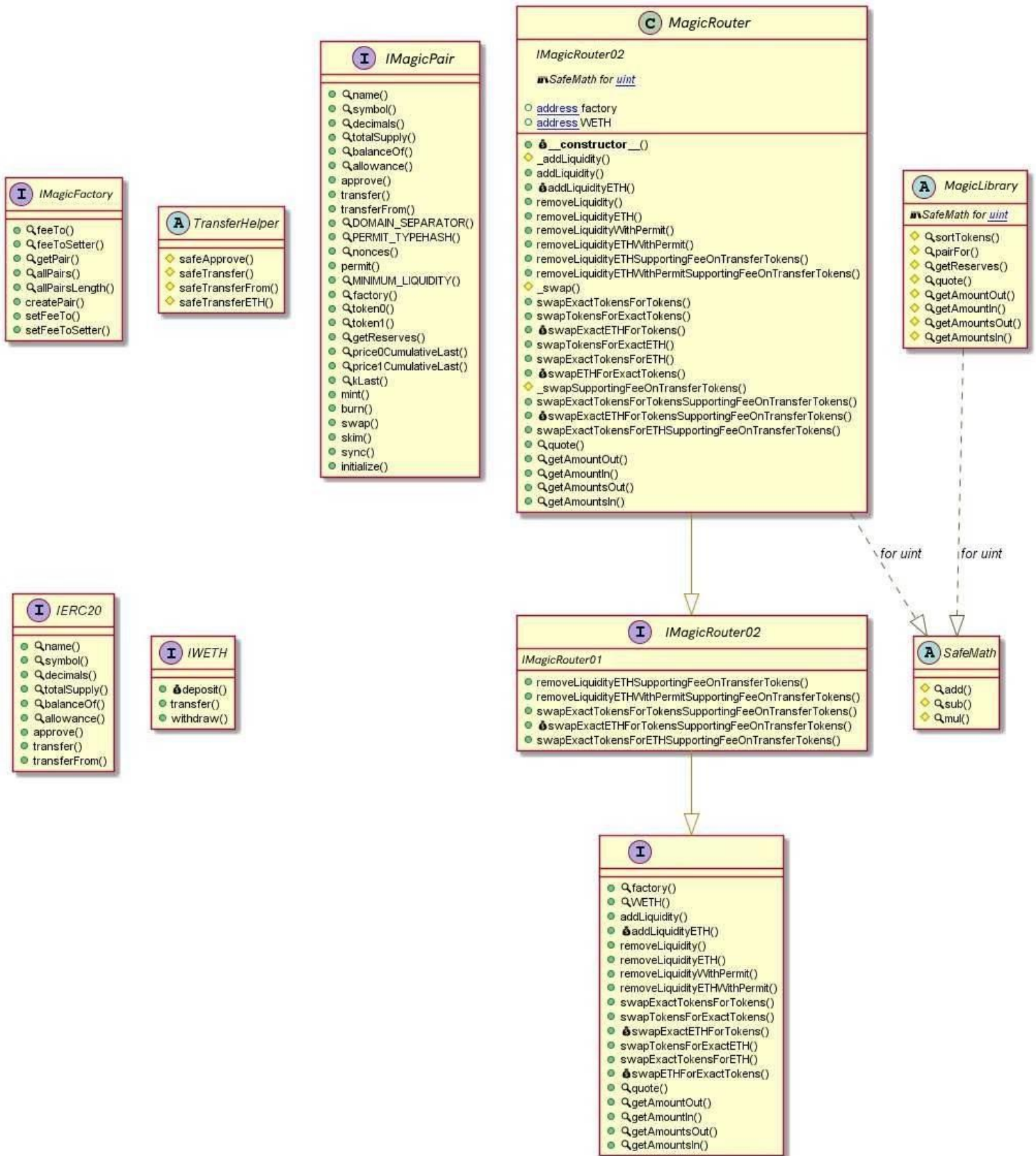
Code Flow Diagram - Magic Protocol MagicToken Diagram



MagicFactory Diagram



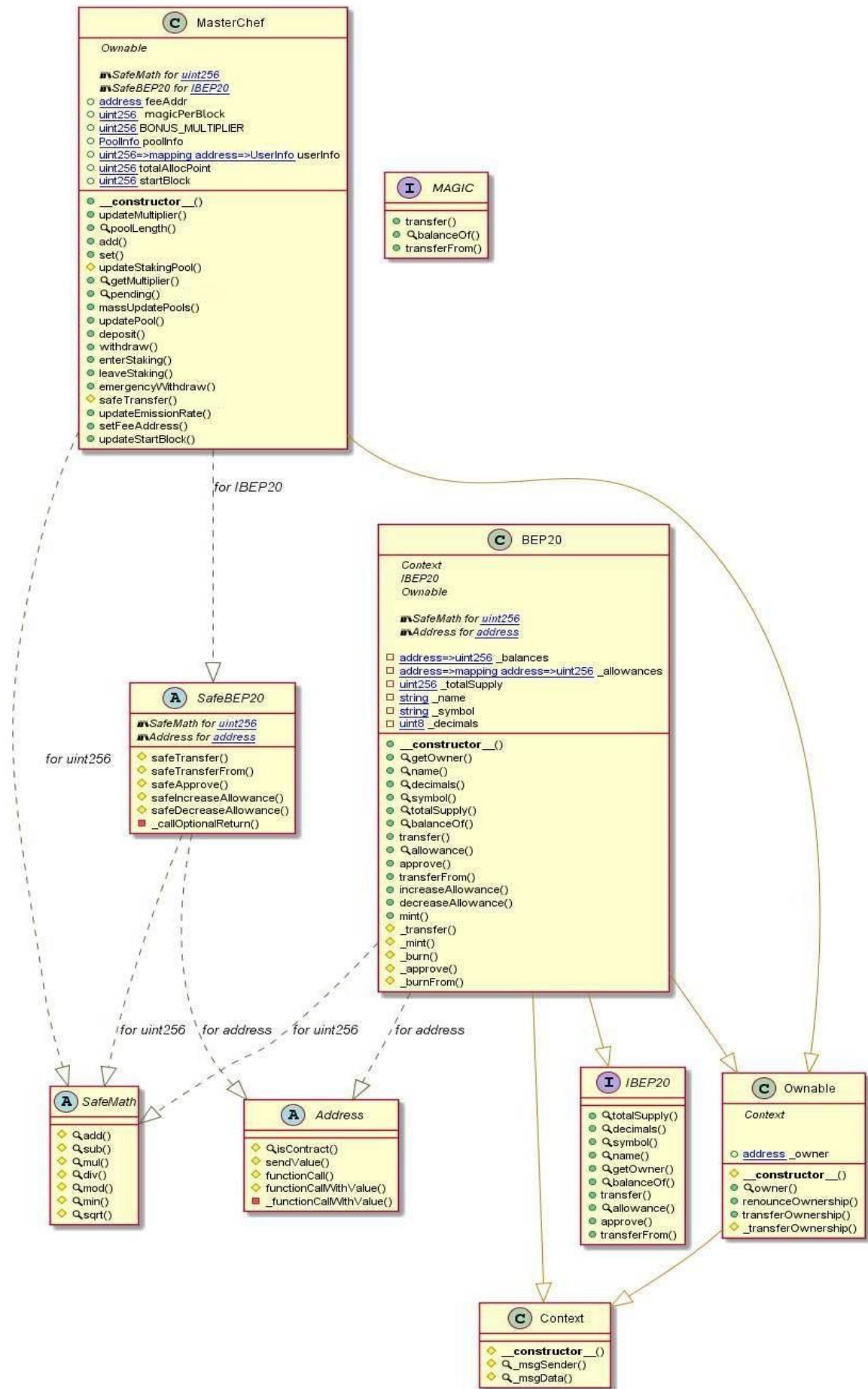
MagicRouter Diagram



This is a private and confidential document. No part of this document should be disclosed to third party without prior written permission of EtherAuthority.

Email: audit@EtherAuthority.io

MasterChef Diagram



This is a private and confidential document. No part of this document should be disclosed to third party without prior written permission of EtherAuthority.

Email: audit@EtherAuthority.io

Solhint Linter

contract.sol

```
Compiler version 0.6.12 does not satisfy the ^0.5.8 semver
requirement
Pos: 1:1
Code contains empty blocks
Pos: 28:18
Use double quotes for string literals
Pos: 41:68
Error message for require is too long
Pos: 9:96
Use double quotes for string literals
Pos: 41:96
Use double quotes for string literals
Pos: 25:225
Use double quotes for string literals
Pos: 26:241
Error message for require is too long
Pos: 9:284
Use double quotes for string literals
Pos: 29:284
Use double quotes for string literals
Pos: 26:302
Use double quotes for string literals
Pos: 26:342
Use double quotes for string literals
Pos: 50:437
Use double quotes for string literals
Pos: 58:440
Error message for require is too long
Pos: 9:441
Use double quotes for string literals
Pos: 26:441
Use double quotes for string literals
Pos: 43:463
Use double quotes for string literals
Pos: 59:496
Error message for require is too long
Pos: 9:511
Use double quotes for string literals
Pos: 49:511
Use double quotes for string literals
Pos: 37:521
Use double quotes for string literals
Pos: 59:693
Use double quotes for string literals
Pos: 69:733
Error message for require is too long
Pos: 9:770
Use double quotes for string literals
Pos: 39:770
Error message for require is too long
Pos: 9:771
```

This is a private and confidential document. No part of this document should be disclosed to third party without prior written permission of EtherAuthority.

Email: audit@EtherAuthority.io

```
Use double quotes for string literals
Pos: 42:771
Use double quotes for string literals
Pos: 59:773
Use double quotes for string literals
Pos: 40:788
Error message for require is too long
Pos: 9:807
Use double quotes for string literals
Pos: 40:807
Use double quotes for string literals
Pos: 61:809
Error message for require is too long
Pos: 9:832
Use double quotes for string literals
Pos: 38:832
Error message for require is too long
Pos: 9:833
Use double quotes for string literals
Pos: 40:833
Use double quotes for string literals
Pos: 60:850
Use double quotes for string literals
Pos: 31:856
Use double quotes for string literals
Pos: 41:856
Error message for require is too long
Pos: 9:965
Error message for require is too long
Pos: 9:966
Error message for require is too long
Pos: 9:967
Avoid making time-based decisions in your business logic
Pos: 17:967
Error message for require is too long
Pos: 9:997
Avoid using inline assembly. It is acceptable only in rare cases
Pos: 9:1089
```

MagicFactory.sol

```
Function name must be in mixedCase
Pos: 5:38
Function name must be in mixedCase
Pos: 5:39
Function name must be in mixedCase
Pos: 5:56
Function name must be in mixedCase
Pos: 5:89
Function name must be in mixedCase
Pos: 5:90
Use double quotes for string literals
Pos: 35:99
Use double quotes for string literals
Pos: 35:103
```

Use double quotes for string literals
Pos: 49:107
Constant name must be in capitalized SNAKE_CASE
Pos: 5:114
Use double quotes for string literals
Pos: 35:114
Constant name must be in capitalized SNAKE_CASE
Pos: 5:115
Use double quotes for string literals
Pos: 37:115
Constant name must be in capitalized SNAKE_CASE
Pos: 5:116
Variable name must be in mixedCase
Pos: 5:121
Avoid using inline assembly. It is acceptable only in rare cases
Pos: 9:131
Use double quotes for string literals
Pos: 27:136
Use double quotes for string literals
Pos: 33:138
Avoid making time-based decisions in your business logic
Pos: 29:187
Use double quotes for string literals
Pos: 46:187
Use double quotes for string literals
Pos: 17:190
Use double quotes for string literals
Pos: 78:196
Explicitly mark visibility of state
Pos: 5:226
Use double quotes for string literals
Pos: 63:264
Use double quotes for string literals
Pos: 32:280
Avoid using low level calls.
Pos: 45:293
Use double quotes for string literals
Pos: 76:294
Use double quotes for string literals
Pos: 40:315
Use double quotes for string literals
Pos: 69:324
Avoid making time-based decisions in your business logic
Pos: 40:325
Error message for require is too long
Pos: 9:375
Use double quotes for string literals
Pos: 32:375
Error message for require is too long
Pos: 9:396
Use double quotes for string literals
Pos: 45:396
Error message for require is too long
Pos: 9:410
Use double quotes for string literals
Pos: 51:410
Use double quotes for string literals
Pos: 67:412
Use double quotes for string literals

```
Pos: 49:419
Error message for require is too long
Pos: 9:428
Use double quotes for string literals
Pos: 49:428
Use double quotes for string literals
Pos: 104:432
Use double quotes for string literals
Pos: 35:473
Use double quotes for string literals
Pos: 39:475
Use double quotes for string literals
Pos: 56:476
Avoid using inline assembly. It is acceptable only in rare cases
Pos: 9:479
Use double quotes for string literals
Pos: 44:490
Use double quotes for string literals
Pos: 44:495
```

MagicRouter.sol

```
Compiler version =0.6.6 does not satisfy the ^0.5.8 semver
requirement
Pos: 1:5
Use double quotes for string literals
Pos: 13:34
Use double quotes for string literals
Pos: 13:47
Use double quotes for string literals
Pos: 13:61
Use double quotes for string literals
Pos: 26:67
Use double quotes for string literals
Pos: 35:260
Use double quotes for string literals
Pos: 35:264
Use double quotes for string literals
Pos: 49:268
Use double quotes for string literals
Pos: 35:277
Use double quotes for string literals
Pos: 39:279
Use double quotes for string literals
Pos: 30:304
Use double quotes for string literals
Pos: 47:305
Use double quotes for string literals
Pos: 31:311
Use double quotes for string literals
Pos: 50:312
Use double quotes for string literals
Pos: 32:321
Use double quotes for string literals
Pos: 50:322
```

```
Use double quotes for string literals
Pos: 35:330
Use double quotes for string literals
Pos: 35:341
Use double quotes for string literals
Pos: 46:380
Use double quotes for string literals
Pos: 55:412
Use double quotes for string literals
Pos: 55:417
Use double quotes for string literals
Pos: 40:478
Use double quotes for string literals
Pos: 40:479
Use double quotes for string literals
Pos: 62:593
Use double quotes for string literals
Pos: 44:607
Use double quotes for string literals
Pos: 34:621
Use double quotes for string literals
Pos: 62:623
Use double quotes for string literals
Pos: 48:635
Use double quotes for string literals
Pos: 44:637
Use double quotes for string literals
Pos: 48:652
Use double quotes for string literals
Pos: 62:654
Use double quotes for string literals
Pos: 34:670
Use double quotes for string literals
Pos: 42:672
Use double quotes for string literals
Pos: 13:714
Use double quotes for string literals
Pos: 34:729
Use double quotes for string literals
Pos: 13:737
Use double quotes for string literals
Pos: 48:752
Use double quotes for string literals
Pos: 44:758
```

MasterChef.sol

```
Compiler version 0.6.12 does not satisfy the ^0.5.8 semver
requirement
Pos: 1:10
Use double quotes for string literals
Pos: 25:40
Use double quotes for string literals
Pos: 26:56
Use double quotes for string literals
```

```
Pos: 29:99
Use double quotes for string literals
Pos: 26:117
Use double quotes for string literals
Pos: 26:157
Use double quotes for string literals
Pos: 50:347
Use double quotes for string literals
Pos: 58:350
Use double quotes for string literals
Pos: 26:351
Use double quotes for string literals
Pos: 43:373
Use double quotes for string literals
Pos: 59:406
Use double quotes for string literals
Pos: 49:421
Use double quotes for string literals
Pos: 37:431
Use double quotes for string literals
Pos: 13:503 Use double quotes for string literals
Pos: 41:641
Use double quotes for string literals
Pos: 59:796 Use double quotes for string literals
Pos: 59:876
Use double quotes for string literals
Pos: 40:891
Use double quotes for string literals
Pos: 40:910
Use double quotes for string literals
Pos: 61:912
Use double quotes for string literals
Pos: 38:935
Use double quotes for string literals
Pos: 40:936
Use double quotes for string literals
Pos: 60:953
Compiler version ^0.6.0 does not satisfy the ^0.5.8 semver
requirement
Pos: 1:958
Use double quotes for string literals
Pos: 29:1153
Use double quotes for string literals
Pos: 29:1190
```

Software analysis result:

These software reported many false positive results and some are informational issues.
So, those issues can be safely ignored.

